SISTEMAS DISTRIBUÍDOS

Microsserviços e API Gateways, Teoria de Desacoplamento

22 DE MARÇO DE 2026

PROF. DR. LINCOLN SPOSITO
Universidade São Judas Tadeu – São Bernardo do Campo

Sumário

1. Fundamentos de Microsserviços 3
Exercícios de Fixação 4
Questões de Múltipla Escolha 4
Questões Descritivas 5
Gabarito e Justificativas 5
Múltipla Escolha 5
Questões Descritivas (Respostas Aceitáveis) 5
2. Teoria de Desacoplamento 6
Exercícios de Fixação 8
Questões de Múltipla Escolha 8
Questões Descritivas 9
Gabarito e Justificativa 9
Múltipla Escolha 9
Questões Descritivas (Respostas Aceitáveis) 9
3. API Gateways: A Porta de Entrada 10
Exercícios de Fixação 12
Questões de Múltipla Escolha 12
Questões Descritivas 12
Gabarito e Justificativa 13
Múltipla Escolha 13
Questões Descritivas (Respostas Aceitáveis) 13
4. Prática de Laboratório (Hands-on Lab) 13
Exercícios de Fixação 16
Questões de Múltipla Escolha 16
Questões Descritivas 17
Gabarito e Justificativa 17
Múltipla Escolha 17
Questões Descritivas (Respostas Aceitáveis) 17
Exercício prático – Desafio: laboratório de **Sistemas Distribuídos** 18
Pré-requisitos para o ambiente do Laboratório de Redes e Cloud 18
1. Pré-requisitos de Software (A Stack de Virtualização) 18
2. Pré-requisitos de Hardware (Mínimo para Estabilidade) 18
Exercícios de Fixação 18

Gabarito e Justificativas 18
Ambiente no Laboratório de Redes e Cloud 18
1. Verificação de Pré-requisitos 18
2. Estruturação do Workspace 18
3. Construção das Imagens (Build) 18
4. Execução e Redes Virtuais 18
Exercícios de Fixação 18
Gabarito e Justificativas 18
Arquivos de configuração (Dockerfiles) 18
Arquivos de Configuração (Boilerplate) 18
1. Microsserviço de Agendamento (Python) 18
2. Microsserviço de Notificação (Node.js) 19
3. Orquestrador (Docker Compose) 19
Guia de Execução para o Aluno 20
Roteiro de laboratório prático em Docker 20
Lab: Decomposição e Isolamento com Docker 20
Exercícios de Fixação (Prática) 21
Gabarito e Justificativa 22

Microsserviços e API Gateways, Teoria de Desacoplamento

Com base no plano de ensino para a disciplina de **Sistemas Distribuídos** da Universidade São Judas Tadeu , abaixo seguem os postos-chave conceituais para a aula de **23/03**, que foca em **Microsserviços, API Gateways e Teoria de Desacoplamento**. Esta aula é considerada o "coração" da ementa, iniciando um bloco de quatro semanas dedicado a preparar os alunos para a separação de responsabilidades em sistemas modernos.

---

1. Fundamentos de Microsserviços

O foco inicial deve ser a transição da mentalidade de sistemas únicos para sistemas modulares.

- **De Monólitos a Microsserviços:** Compreender as limitações de escalabilidade e manutenção de arquiteturas monolíticas versus as vantagens da componentização.
- **Independência de Deploy:** Cada serviço deve ser capaz de ser atualizado, implantado e escalado de forma independente.
- **Poliglotismo de Stack:** A possibilidade de utilizar diferentes linguagens ou bancos de dados para diferentes serviços, conforme a necessidade técnica.

Prezado aluno, para darmos início ao nosso mergulho em **Sistemas Distribuídos**, é fundamental compreendermos a mudança de paradigma que sustenta as arquiteturas modernas. Este material foi elaborado para que você domine a transição da mentalidade de sistemas únicos para ecossistemas modulares, focando nos pilares de microsserviços que discutiremos em nossos laboratórios.

De Monólitos a Microsserviços representa a evolução da engenharia de software onde deixamos de construir aplicações como uma unidade única e indivisível (monolítica), em que interface, lógica de negócio e acesso a dados compartilham o mesmo processo e base de código, para adotar uma estrutura de componentes independentes. Em um monólito, uma pequena alteração em um módulo de pagamento pode exigir o re-deploy de todo o sistema de catálogo e logística, gerando riscos desnecessários e lentidão. Ao migrarmos para microsserviços, cada funcionalidade torna-se um serviço autônomo que se comunica via protocolos leves como HTTP/REST.

Exemplo Teórico: Imagine uma plataforma de e-commerce; no modelo monolítico, se o módulo de busca travar por excesso de requisições, ele pode consumir toda a memória do servidor e derrubar o carrinho de compras. No modelo de microsserviços, a busca é um serviço isolado; se ele falhar ou precisar de mais recursos, os outros serviços continuam operacionais.

Exemplo Prático: Uma aplicação de streaming de vídeo que separa o "Gerenciamento de Usuários" do "Processamento de Vídeo". Durante o lançamento de uma série popular, apenas o serviço de vídeo precisa ser escalado horizontalmente em instâncias adicionais de nuvem, sem desperdiçar recursos com o módulo de cadastro de usuários.

Independência de Deploy é a capacidade técnica de colocar um serviço em produção sem depender da coordenação ou do ciclo de vida de outros serviços do ecossistema. Isso é alcançado através do baixo acoplamento, onde as interfaces de comunicação (APIs) são estáveis e bem definidas. Essa autonomia permite que equipes diferentes trabalhem em ritmos distintos, utilizando ferramentas de CI/CD (Integração e Entrega Contínua) para acelerar a inovação.

Exemplo Teórico: Se a equipe de logística decide implementar um novo algoritmo de cálculo de frete, ela pode realizar o deploy dessa nova versão do serviço de "Frete" às 14h de uma terça-feira sem precisar avisar a equipe que cuida do "Inventário", desde que o contrato da API não seja quebrado.

Exemplo Prático: Em um sistema bancário distribuído, a atualização do serviço de "Notificações Push" para suportar um novo formato de mensagem pode ser feita instantaneamente via containers Docker, garantindo que o núcleo de "Transações" permaneça intocado e seguro durante todo o processo.

Poliglotismo de Stack refere-se à liberdade arquitetural de escolher a ferramenta mais adequada para o problema específico de cada serviço, em vez de forçar uma única linguagem ou banco de dados para toda a empresa. Como a comunicação é feita por protocolos padrão (JSON/HTTP), o que acontece "dentro" do serviço é irrelevante para quem o consome.

Exemplo Teórico: Um serviço que exige processamento matemático intenso e concorrência pode ser escrito em Go ou C++, enquanto um serviço de interface simples pode ser feito em Node.js, e ambos convivem perfeitamente no mesmo sistema distribuído.

Exemplo Prático: Em um sistema de redes sociais, o serviço de "Perfil de Usuário" pode utilizar um banco de dados relacional (SQL) para garantir consistência de dados, enquanto o serviço de "Feed de Notícias" utiliza um banco NoSQL (como MongoDB ou Redis) para garantir a alta velocidade de leitura e escalabilidade necessária para milhões de postagens.

Exercícios de Fixação

Questões de Múltipla Escolha

1. **Qual das alternativas abaixo descreve a principal vantagem da Independência de Deploy em uma arquitetura de microsserviços?**
    - a) A garantia de que todos os serviços utilizem a mesma versão de biblioteca de segurança.
    - b) A capacidade de atualizar uma funcionalidade específica sem interromper ou re-implantar o restante do sistema.
    - c) A redução obrigatória do tráfego de rede entre os componentes do sistema.
    - d) A eliminação total da necessidade de utilizar APIs para comunicação entre serviços.
2. **O conceito de "Poliglotismo de Stack" permite que:**
    - a) O sistema utilize apenas uma linguagem de programação para facilitar a manutenção centralizada.

- b) Cada serviço seja desenvolvido com a tecnologia (linguagem/banco de dados) mais eficiente para sua função específica.
- c) Os dados de todos os serviços sejam armazenados obrigatoriamente em um único banco de dados central.
- d) O protocolo HTTP seja substituído por linguagens de tradução simultânea entre servidores.

Questões Descritivas

1. Explique como a transição de uma arquitetura monolítica para microsserviços impacta a **tolerância a falhas** de um sistema distribuído, utilizando o conceito de isolamento de processos.
2. Imagine que você é o arquiteto de um sistema de saúde. Justifique a aplicação do **Poliglotismo de Stack** para um módulo de "Análise de Imagens de Raio-X" e um módulo de "Agendamento de Consultas".

Gabarito e Justificativas

Múltipla Escolha

1. **Resposta correta: b)**
   - *Justificativa:* A independência de deploy visa justamente o desacoplamento do ciclo de vida dos serviços. Ao contrário do monólito, onde o deploy é uma unidade, nos microsserviços cada componente tem seu próprio pipeline, aumentando a agilidade e reduzindo o risco de efeitos colaterais em módulos não relacionados.
2. **Resposta correta: b)**
   - *Justificativa:* O poliglotismo de stack é a quebra da "ditadura tecnológica". Ele permite que problemas de processamento paralelo sejam resolvidos com linguagens performáticas, enquanto problemas de volume de dados sejam resolvidos com bancos NoSQL específicos, desde que a interface de saída (API) seja padronizada.

Questões Descritivas (Respostas Aceitáveis)

1. **Resposta esperada:** O aluno deve explicar que, em um monólito, uma falha de memória ou erro crítico em qualquer parte do código pode derrubar o processo inteiro da aplicação (falha total). Nos microsserviços, como cada serviço roda em seu próprio container ou processo isolado, uma falha no serviço A não se propaga diretamente para o serviço B. O sistema como um todo permanece funcional (degradação graciosa), permitindo que apenas o componente falho seja reiniciado ou escalado.
2. **Resposta esperada:** O aluno deve argumentar que o módulo de "Análise de Imagens" exige alto poder computacional e bibliotecas específicas de IA, sendo melhor desenvolvido em linguagens como Python ou C++ com suporte a GPU e bancos de dados de objetos/arquivos. Já

o "Agendamento" lida com transações críticas e consistência de horários, sendo mais adequado o uso de Java ou C# com um banco de dados relacional (SQL) para garantir as propriedades ACID das consultas.

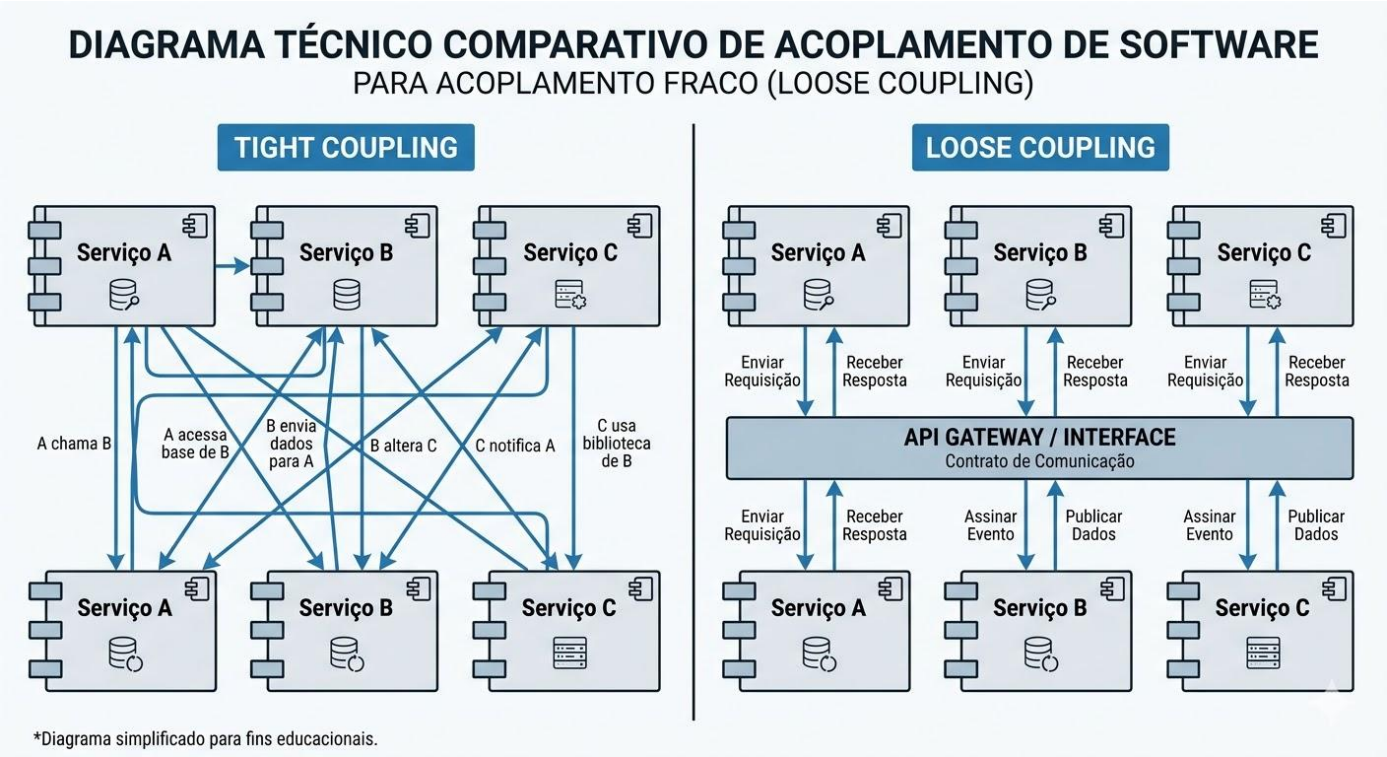
2. Teoria de Desacoplamento

Este é o conceito central para garantir que a falha em um serviço não derrube todo o ecossistema.

- **Acoplamento Fraco (Loose Coupling):** Redução das dependências diretas entre componentes para facilitar a manutenção e evolução do sistema.
- **Alta Coesão:** Garantir que cada microsserviço tenha uma responsabilidade única e bem definida.
- **Comunicação Assíncrona:** Introdução preliminar à ideia de que serviços não precisam esperar uma resposta imediata para continuar operando (preparando o terreno para a aula seguinte de mensageria).

Dando continuidade ao nosso programa de **Sistemas Distribuídos**, avançamos agora para o núcleo estratégico da arquitetura moderna: a **Teoria de Desacoplamento**. Compreender este capítulo é o que diferencia um desenvolvedor de software de um arquiteto de sistemas capaz de projetar soluções resilientes e globais.

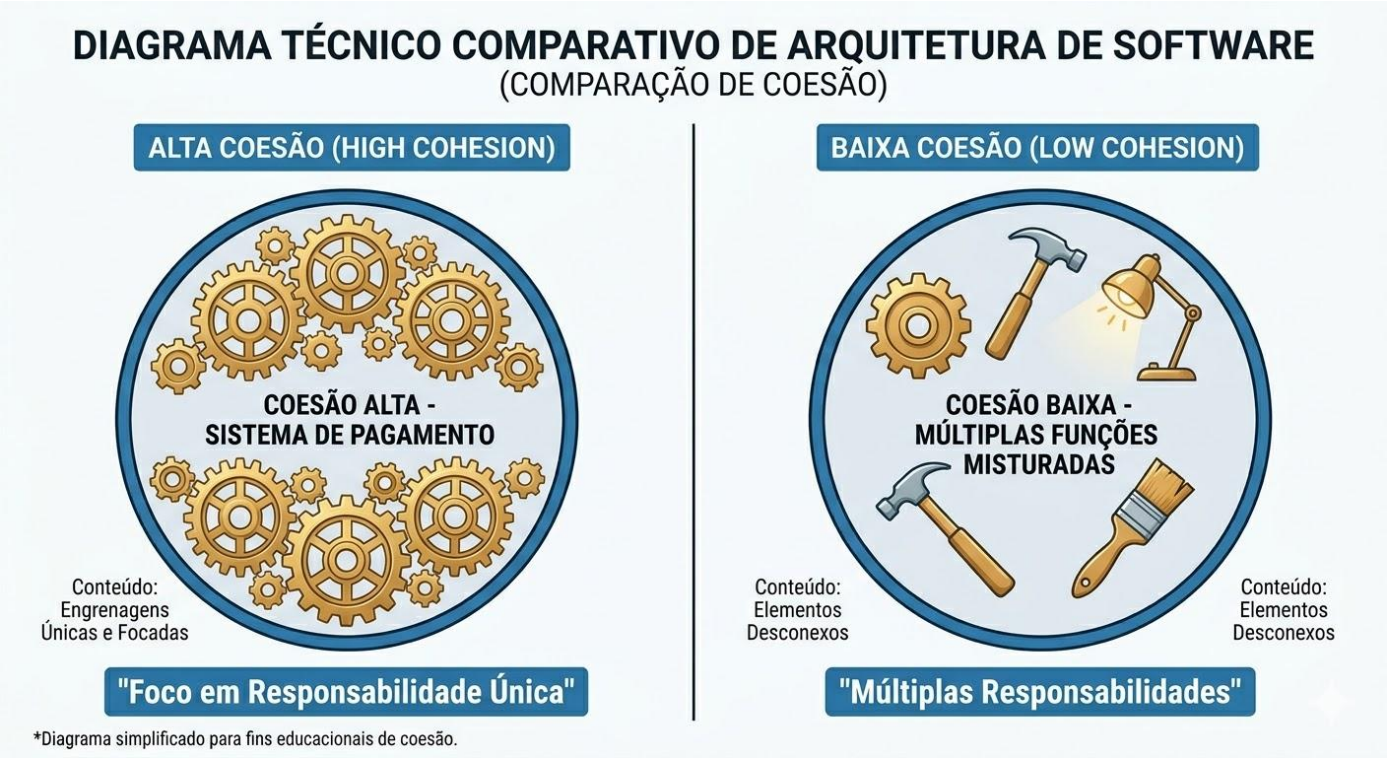
Acoplamento Fraco (Loose Coupling) define o grau de interdependência entre os módulos de um sistema, sendo o objetivo primordial de qualquer arquitetura distribuída reduzir essas conexões ao mínimo necessário. Em sistemas com acoplamento forte, uma mudança na estrutura de uma tabela do banco de dados de um serviço pode "quebrar" outros dez serviços que dependem dessa estrutura interna. O acoplamento fraco dita que os componentes devem interagir apenas através de contratos de interface (APIs) bem definidos, sem conhecer os detalhes de implementação uns dos outros.



Exemplo Teórico: Imagine dois componentes onde o Serviço A chama o Serviço B. No acoplamento fraco, o Serviço A não sabe se o Serviço B foi escrito em Java ou se usa um banco Oracle; ele apenas envia um JSON e recebe uma resposta padronizada.

Exemplo Prático: Em um sistema de pagamentos, o módulo de "Checkout" não acessa diretamente o banco de dados do módulo de "Estoque". Ele envia uma requisição para a API de Estoque perguntando "tem o item X?". Se o Estoque decidir mudar seu banco de dados de MySQL para MongoDB, o Checkout nem sequer perceberá a mudança, pois a interface da API permanece a mesma.

Alta Coesão é o princípio que garante que cada microsserviço tenha uma responsabilidade única, clara e bem delimitada, seguindo o Princípio da Responsabilidade Única (SRP). Um serviço coeso é aquele em que todas as suas funções internas estão intimamente relacionadas ao seu propósito central. Quando a coesão é baixa, temos serviços "faz-tudo", que são difíceis de testar, manter e escalar, pois misturam lógicas de domínios diferentes.

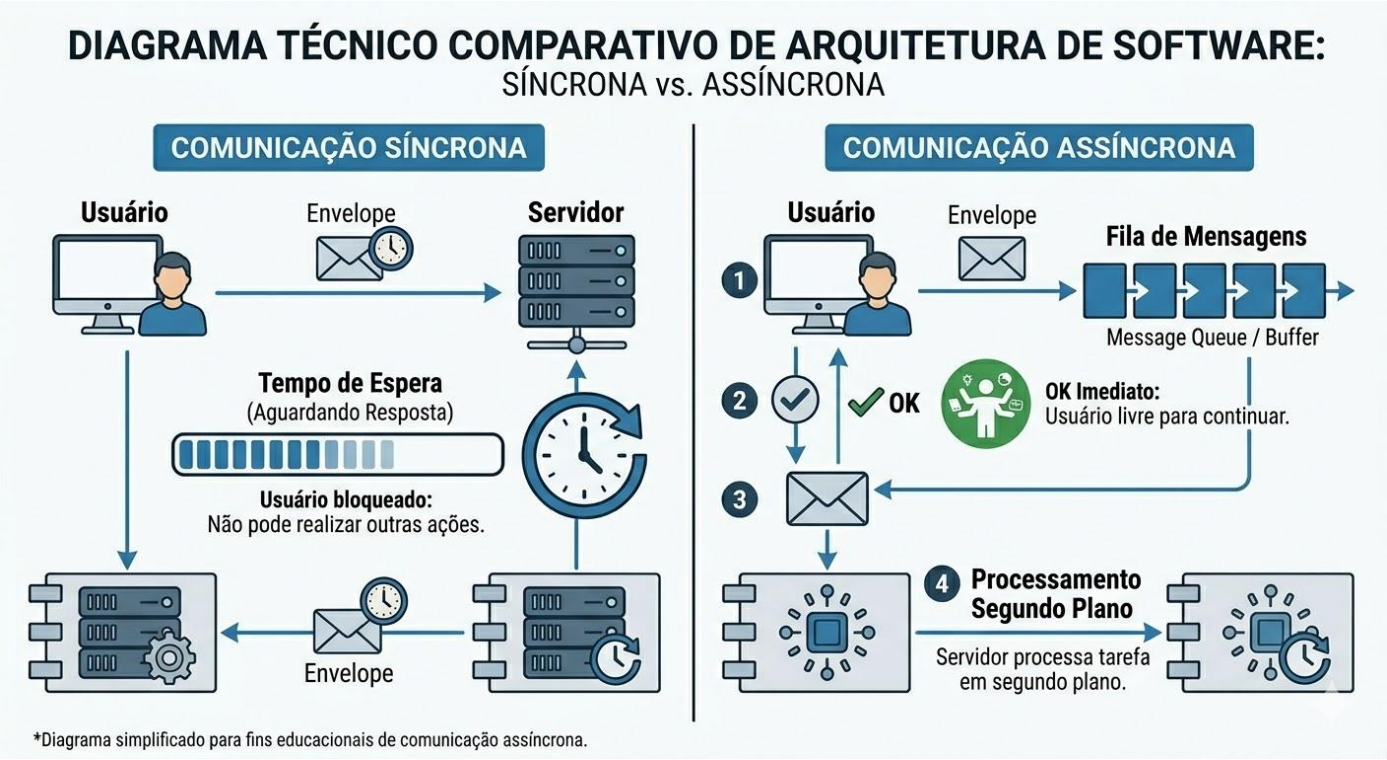


Exemplo Teórico: Um microsserviço de "Autenticação" deve lidar apenas com logins, tokens e permissões. Se ele começar a calcular o valor do frete dos pedidos, ele perde sua coesão, tornando-se um "mini-monólito".

Exemplo Prático: Em um aplicativo de entrega de comida, o serviço de "Avaliações" cuida apenas de notas e comentários. Se precisarmos escalar o sistema porque o número de avaliações cresceu absurdamente, escalamos apenas esse serviço. Se a lógica de avaliações estivesse misturada com a lógica de "Processamento de Pedidos", teríamos que gastar recursos escalando o processamento de pedidos desnecessariamente.

Comunicação Assíncrona é a técnica de integração onde o emissor de uma mensagem não fica bloqueado aguardando uma resposta imediata para continuar seu processamento. Em sistemas distribuídos de alta escala, esperar por uma resposta (síncrono) é um risco, pois se o serviço receptor

estiver lento, o emissor também ficará gerando um efeito cascata de lentidão. A comunicação assíncrona permite que o sistema seja mais resiliente a picos de carga e falhas temporárias.



Exemplo Teórico: O conceito de "disparar e esquecer" (fire-and-forget) ou o uso de callbacks. O serviço envia a tarefa para uma fila e fica livre para atender o próximo cliente, enquanto o processamento ocorre em background.

Exemplo Prático: Quando você solicita a "Recuperação de Senha" em um site, o sistema não faz você esperar na tela até que o e-mail seja efetivamente entregue pelo servidor de SMTP. O serviço de conta apenas envia um comando para o serviço de e-mail e responde instantaneamente para você: "Instruções enviadas!". O envio real do e-mail acontece de forma assíncrona segundos depois.

Exercícios de Fixação

Questões de Múltipla Escolha

1. **Em uma arquitetura de microsserviços, o que acontece quando aumentamos o acoplamento entre os serviços?**
    - a) O sistema torna-se mais fácil de escalar, pois os serviços estão mais unidos.
    - b) A manutenção torna-se mais complexa, pois uma alteração em um serviço pode exigir alterações em vários outros.
    - c) A performance da rede aumenta, já que os serviços não precisam de APIs para conversar.
    - d) O sistema torna-se mais resiliente a falhas, pois os serviços dependem uns dos outros.

2. **Qual a principal característica de um serviço que possui "Alta Coesão"?**
    - a) Ele executa múltiplas funções de diferentes domínios do negócio para economizar recursos.
    - b) Ele depende fortemente de como os outros serviços foram programados.
    - c) Ele foca em uma única responsabilidade ou funcionalidade lógica do sistema.
    - d) Ele obriga que a comunicação seja sempre síncrona para garantir a entrega.

Questões Descritivas

1. Explique a relação entre **Acoplamento Fraco** e a **Tolerância a Falhas**. Como o isolamento entre serviços impede que um erro no serviço de "Recomendações" derrube o serviço de "Pagamentos"?
2. Descreva um cenário em que a **Comunicação Síncrona** (esperar a resposta) seria prejudicial para a experiência do usuário em um aplicativo de grande escala, e como a **Comunicação Assíncrona** resolveria esse problema.

Gabarito e Justificativa

Múltipla Escolha

1. **Resposta correta: b)**
    - *Justificativa:* O acoplamento mede a dependência. Se o acoplamento é alto, os serviços perdem sua autonomia. Isso cria o chamado "Monólito Distribuído", onde você tem os problemas de rede dos microsserviços e a dificuldade de alteração dos monólitos.
2. **Resposta correta: c)**
    - *Justificativa:* A alta coesão permite que o serviço seja uma "caixa preta" especialista em um assunto. Isso facilita o entendimento do código, a realização de testes unitários e a escalabilidade granular.

Questões Descritivas (Respostas Aceitáveis)

1. **Resposta esperada:** O aluno deve explicar que com o acoplamento fraco, o serviço de Pagamentos não conhece a lógica interna do serviço de Recomendações. Se o serviço de Recomendações falhar, o serviço de Pagamentos pode simplesmente receber um erro de *timeout* ou uma resposta vazia e continuar seu fluxo principal (conclusão da venda) sem ser interrompido, pois ele não é dependente vital daquela resposta para sua função primária.
2. **Resposta esperada:** O aluno pode citar o exemplo de um upload de um arquivo pesado ou processamento de um relatório. Se for síncrono, a tela do usuário trava até o fim do processo. Se for assíncrono, o usuário recebe um aviso de "Processando" e pode continuar navegando no app, enquanto um serviço de background cuida da tarefa pesada, melhorando a percepção de velocidade e disponibilidade.

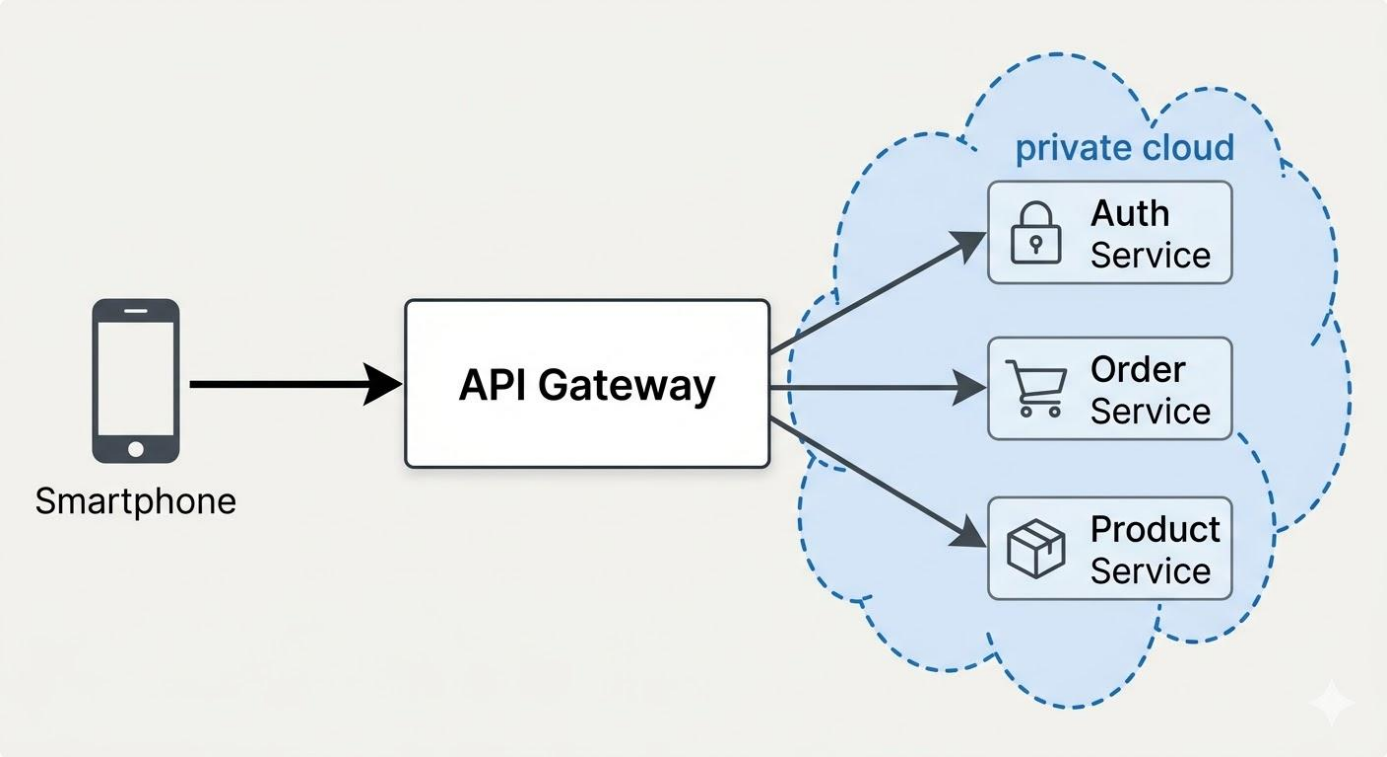
3. API Gateways: A Porta de Entrada

Como o sistema se apresenta para o mundo exterior de forma unificada.

- **Ponto Único de Entrada:** O papel do Gateway em rotear requisições para os microsserviços internos corretos.
- **Agregação de Requisições:** Reduzir o número de chamadas do cliente ao combinar respostas de múltiplos serviços em uma única resposta de API.
- **Segurança e Cross-Cutting Concerns:** Centralização de autenticação (OAuth2/JWT), autorização, limites de taxa (rate limiting) e log de acessos.

Para encerrarmos este módulo sobre a estrutura de sistemas modernos, vamos analisar como os componentes que aprendemos a isolar se comunicam com o usuário final. O **API Gateway** funciona como o maestro dessa orquestra, sendo a peça-chave para a organização e segurança de qualquer ecossistema distribuído.

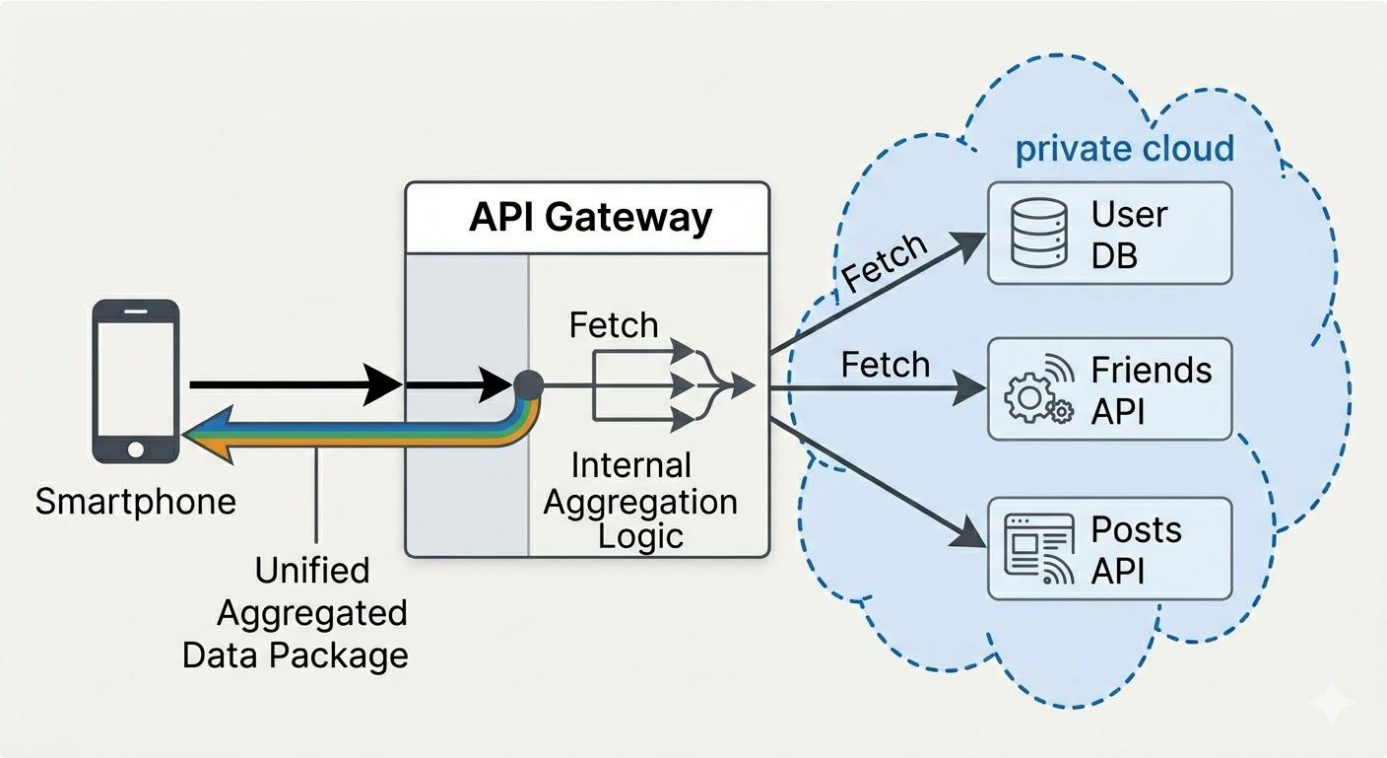
Ponto Único de Entrada é a função primordial de um Gateway, atuando como um roteador inteligente que oculta a complexidade interna da infraestrutura. Em uma arquitetura de microsserviços, cada serviço possui seu próprio endereço IP e porta; sem um Gateway, o cliente (como um aplicativo móvel) precisaria conhecer e gerenciar dezenas de conexões diferentes, o que é inviável e inseguro. O Gateway recebe todas as chamadas em uma única URL pública e as direciona internamente para o serviço correto.



Exemplo Teórico: Imagine um grande hospital onde o paciente não precisa saber em qual sala cada médico está; ele se dirige à recepção principal (o Gateway), e a recepcionista o encaminha para o especialista correto com base no seu prontuário.

Exemplo Prático: Em um sistema de e-commerce, o frontend envia uma requisição para api.loja.com/produtos e outra para api.loja.com/carrinho. O Gateway intercepta essas chamadas e as roteia internamente: a primeira vai para o contêiner do "Serviço de Catálogo" e a segunda para o "Serviço de Vendas", sem que o usuário saiba que são servidores distintos.

Agregação de Requisições resolve o problema da "conversação excessiva" entre cliente e servidor, permitindo que o Gateway combine dados de diversos serviços em uma única resposta. Em dispositivos móveis, onde a latência de rede é um fator crítico, disparar cinco chamadas HTTP para montar uma única tela consome bateria e largura de banda. O Gateway atua como um agregador: ele dispara essas chamadas internamente (na rede local de alta velocidade do data center) e devolve um único pacote JSON consolidado.



Exemplo Teórico: Pense em um serviço de buffet de festas. Em vez de você ligar individualmente para o fornecedor de salgados, o de doces e o de bebidas, você faz um único pedido ao organizador, que consolida tudo e entrega a festa pronta no horário combinado.

Exemplo Prático: Ao abrir a tela de "Perfil do Usuário" em uma rede social, o Gateway consulta simultaneamente o "Serviço de Dados Cadastrais", o "Serviço de Amigos" e o "Serviço de Posts Recentes". Ele agrupa essas três respostas em um único objeto de dados e o envia ao smartphone do usuário em uma única transação de rede.

Segurança e Cross-Cutting Concerns refere-se à centralização de tarefas que são comuns a todos os serviços, como autenticação, autorização e controle de tráfego. Em vez de implementar a verificação de um token JWT ou um limite de 100 acessos por minuto dentro de cada microsserviço individualmente — o que geraria repetição de código e falhas de segurança — essas regras são

aplicadas na "fronteira" do sistema. Se uma requisição não possui um token válido, o Gateway a barra imediatamente, protegendo os serviços internos de processamento desnecessário.

Exemplo Teórico: Considere o controle de acesso de um prédio comercial. O segurança na portaria verifica o crachá (autenticação) e vê se você tem permissão para subir (autorização) antes mesmo de você entrar no elevador para ir a uma sala específica.

Exemplo Prático: Uma API de serviços financeiros utiliza o Gateway para aplicar *Rate Limiting*. Se um bot tentar realizar 500 consultas de saldo por segundo, o Gateway identifica o excesso e bloqueia o IP temporariamente, garantindo que o "Serviço de Contas" não sofra uma sobrecarga ou um ataque de negação de serviço (DDoS).

Exercícios de Fixação

Questões de Múltipla Escolha

1. **Qual é a principal vantagem da "Agregação de Requisições" realizada por um API Gateway?**
    - a) Aumentar a segurança dos dados armazenados nos bancos de dados NoSQL.
    - b) Permitir que o cliente utilize diferentes linguagens de programação em cada tela.
    - c) Reduzir a latência percebida pelo usuário ao diminuir o número de chamadas de rede entre o cliente e o backend.
    - d) Substituir a necessidade de utilizar contêineres Docker para os microsserviços internos.
2. **Sobre a centralização de Segurança (Cross-Cutting Concerns) no Gateway, é correto afirmar que:**
    - a) Ela obriga que cada microsserviço tenha seu próprio sistema de login independente.
    - b) Ela simplifica o desenvolvimento dos microsserviços, pois eles não precisam gerenciar tarefas como validação de tokens e limites de taxa individualmente.
    - c) Ela impede que o sistema utilize comunicação assíncrona entre os serviços.
    - d) Ela serve apenas para registrar logs de acesso, não interferindo na autenticação.

Questões Descritivas

1. Explique como o API Gateway contribui para o conceito de **Independência de Deploy** (visto no tópico 1), considerando que o endereço IP de um microsserviço pode mudar quando ele é atualizado ou escalado.
2. Imagine um cenário onde um sistema de streaming não utiliza um Gateway e o aplicativo móvel se comunica diretamente com 15 microsserviços. Quais seriam os dois principais problemas (técnicos ou de experiência) que este sistema enfrentaria?

Gabarito e Justificativa

Múltipla Escolha

1. **Resposta correta: c)**
    - *Justificativa:* A agregação visa eficiência. Em redes móveis, o custo de abrir uma conexão HTTP é alto. Ao consolidar múltiplas fontes de dados em uma única resposta, o Gateway melhora significativamente a performance da aplicação para o usuário final.
2. **Resposta correta: b)**
    - *Justificativa:* Centralizar preocupações transversais (cross-cutting) no Gateway promove a alta coesão dos microsserviços, permitindo que os desenvolvedores foquem apenas na lógica de negócio, enquanto a infraestrutura cuida da segurança e governança.

Questões Descritivas (Respostas Aceitáveis)

1. **Resposta esperada:** O aluno deve explicar que o Gateway atua como uma camada de abstração (um "endereço fixo"). Quando um microsserviço é atualizado ou movido para outro servidor, seu endereço interno muda, mas o Gateway apenas atualiza sua tabela de rotas interna. Para o mundo exterior (o cliente), a URL permanece a mesma, permitindo deploys transparentes e sem interrupção de serviço.
2. **Resposta esperada:** O aluno deve citar: 1) **Vulnerabilidade de Segurança**, pois 15 serviços diferentes estariam expostos à internet pública, aumentando a superfície de ataque; 2) **Degradação da Experiência do Usuário/Performance**, devido ao alto número de requisições que o dispositivo teria que gerenciar, causando lentidão e alto consumo de dados/bateria.

---

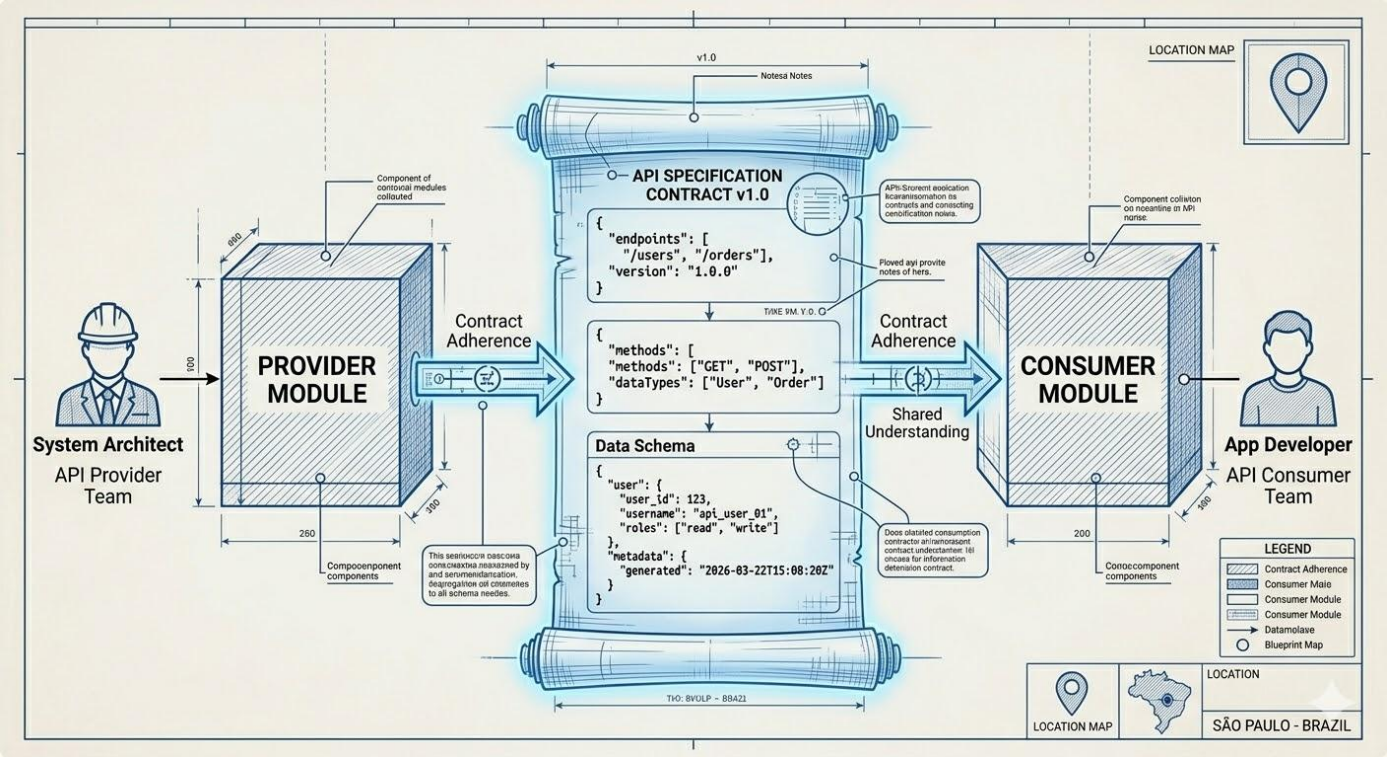
4. Prática de Laboratório (Hands-on Lab)

Seguindo a metodologia de **Laboratório Ativo**, a parte prática deve focar na implementação e integração.

- **Design de Interfaces:** Desenvolver contratos de API claros para que os serviços se comuniquem de forma previsível.
- **Roteamento:** Configuração de um Gateway simples para gerenciar o tráfego entre dois serviços distintos (ex: um serviço de "Usuário" e um de "Notificação").
- **Observabilidade Inicial:** Monitorar como as requisições fluem através do Gateway para os serviços de backend.

Para consolidarmos o aprendizado deste módulo, entramos na fase de **Prática de Laboratório (Hands-on Lab)**. Esta etapa é o momento em que a teoria de microsserviços e gateways ganha vida através da implementação técnica, permitindo que você visualize o comportamento real de um sistema distribuído em funcionamento no nosso Laboratório de Redes e Cloud.

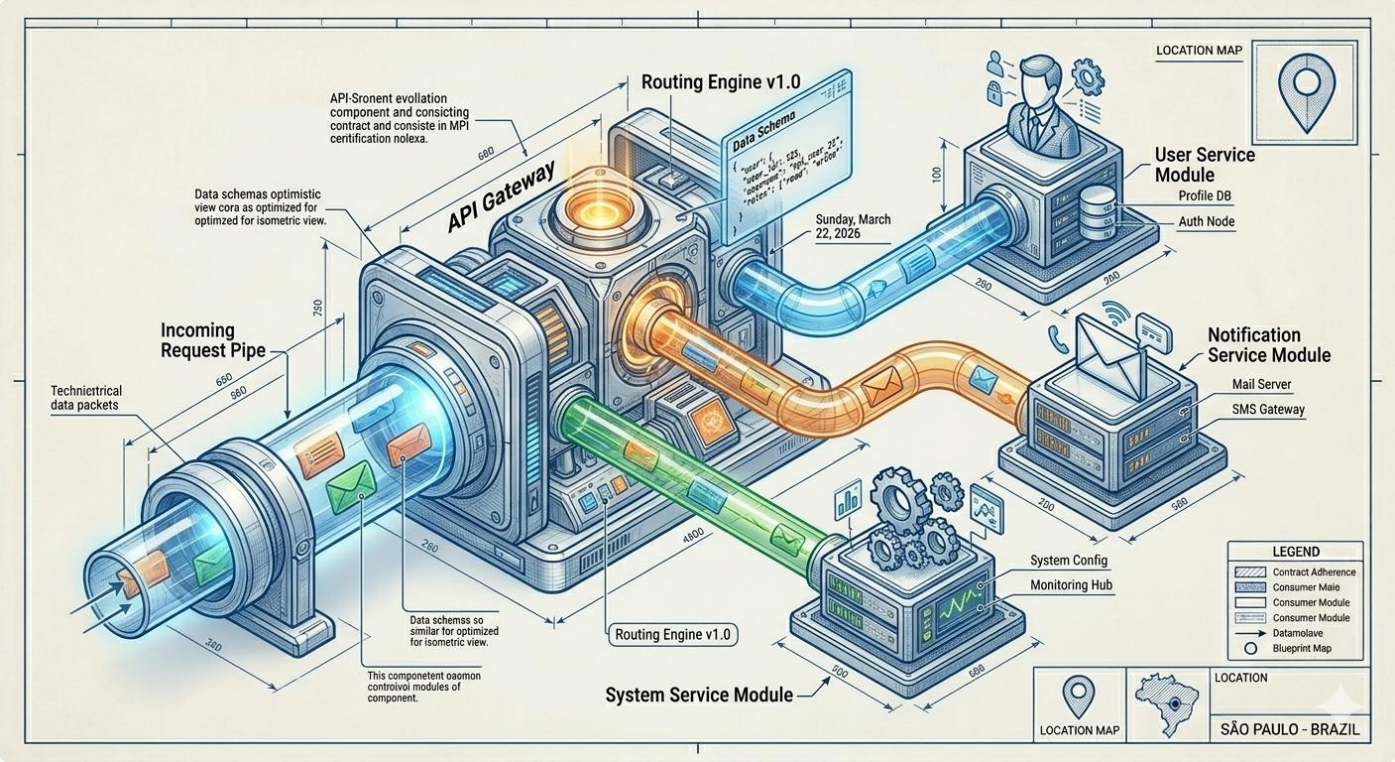
Design de Interfaces constitui a definição rigorosa dos contratos de comunicação, garantindo que diferentes serviços consigam trocar informações de maneira previsível e sem erros de interpretação. Em sistemas distribuídos, o "contrato" é a única verdade compartilhada entre as equipes; se o serviço de "Pedido" espera um campo chamado id_cliente e o serviço de "Usuário" envia uuid, a integração falha. Um bom design utiliza padrões como REST e formatos como JSON, especificando não apenas os dados, mas também os códigos de status HTTP adequados para cada situação.



Exemplo Teórico: Imagine um contrato de aluguel de imóvel; ele define exatamente quem paga, quanto paga e qual a data de vencimento. Se qualquer uma das partes ignorar essas regras (o contrato), o sistema (a moradia) entra em conflito.

Exemplo Prático: Ao projetar a interface do serviço de "Usuário", definimos que o endpoint GET /usuarios/{id} deve retornar obrigatoriamente um objeto JSON contendo nome, email e status_da_conta. Se o desenvolvedor do frontend seguir esse contrato, ele pode construir a interface de perfil mesmo antes do backend estar totalmente pronto.

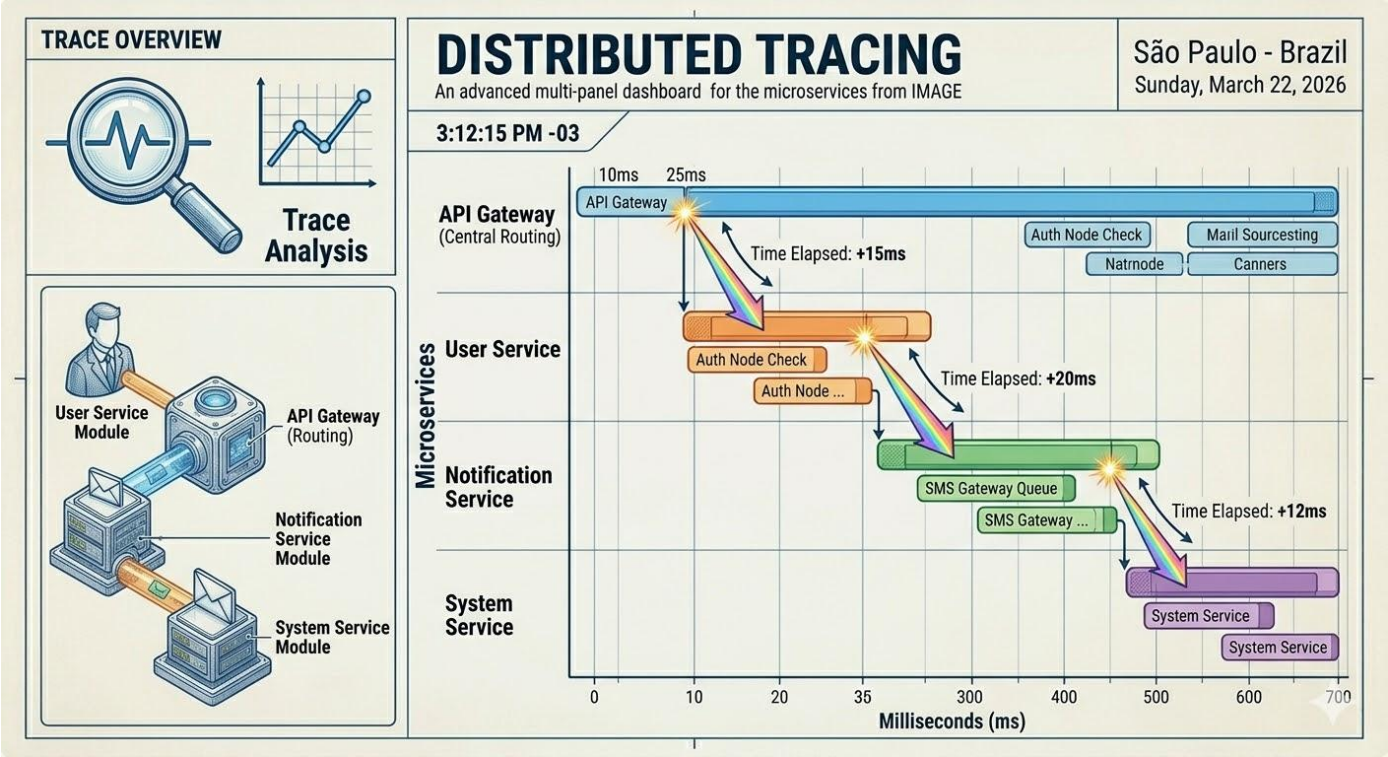
Roteamento é o processo técnico de configuração do Gateway para atuar como o despachante de tráfego, encaminhando cada requisição externa para o seu destino interno específico baseado em regras de URL ou cabeçalhos. Esta configuração é o que permite que o sistema pareça uma unidade coesa para o usuário, enquanto, nos bastidores, o Gateway gerencia o balanço de carga e o endereçamento IP de cada contêiner. Sem o roteamento centralizado, a manutenção de um ecossistema com dezenas de serviços seria impossível de escalar.



Exemplo Teórico: Pense no roteamento como o sistema de triagem de uma central de logística de correios. Todos os pacotes chegam a um único centro de distribuição (o Gateway), que lê a etiqueta de destino (a URL) e decide se o pacote deve ir para o caminhão de entrega local ou para o transporte internacional.

Exemplo Prático: No laboratório, configuramos um Gateway (como o Nginx ou Kong) para que, quando ele receber uma chamada em api.sistema.com/auth, ele direcione o tráfego para o contêiner do serviço de "Usuário" na porta 8080, e quando receber algo em api.sistema.com/notificar, ele redirecione para o serviço de "Notificação" na porta 3000.

Observabilidade Inicial refere-se à capacidade de monitorar e entender o fluxo das requisições conforme elas atravessam as diversas camadas do sistema distribuído. Diferente do monitoramento simples (que apenas diz se o sistema está "vivo" ou "morto"), a observabilidade foca no rastreamento do caminho: quanto tempo o Gateway levou para processar a entrada, qual serviço de backend demorou mais e onde ocorreu um eventual erro. Isso é feito através de logs centralizados e métricas de tempo de resposta.



Exemplo Teórico: É como um sistema de rastreamento de encomendas premium, onde você não sabe apenas que o pacote saiu da loja, mas vê exatamente em qual posto de controle ele está passando, quanto tempo ficou parado em cada cidade e qual funcionário o manipulou.

Exemplo Prático: Durante a prática, utilizamos ferramentas de log no Gateway para visualizar que uma requisição de "Esqueci minha senha" entrou pelo Gateway às 14:05:01, foi encaminhada ao serviço de "Usuário" às 14:05:02 e gerou um trigger para o serviço de "Notificação" às 14:05:05, permitindo identificar gargalos de performance entre os serviços.

Exercícios de Fixação

Questões de Múltipla Escolha

1. **Por que o "Design de Interfaces" é considerado um elemento crítico para a escalabilidade de sistemas distribuídos?**
    - a) Porque ele obriga que todos os serviços sejam escritos na mesma linguagem de programação.
    - b) Porque ele permite que diferentes equipes desenvolvam e atualizem serviços de forma independente, desde que respeitem o contrato estabelecido.
    - c) Porque ele elimina a necessidade de utilizar Gateways no roteamento de mensagens.
    - d) Porque ele garante que o hardware do servidor nunca sofra sobrecarga de memória.
2. **No contexto de Roteamento, qual a função principal do Gateway ao receber uma requisição externa?**
    - a) Recompilar o código fonte do microsserviço para garantir que ele esteja atualizado.

- b) Criptografar o banco de dados do sistema para evitar vazamentos de dados.
- c) Identificar o destino correto da requisição e encaminhá-la para o microsserviço interno correspondente.
- d) Armazenar todos os arquivos estáticos do frontend para reduzir o uso de containers.

Questões Descritivas

1. Descreva a importância da **Observabilidade** em um cenário onde um usuário relata que "o sistema está lento". Como o rastreamento via Gateway ajudaria a identificar o culpado em uma arquitetura de microsserviços?
2. Explique como um erro no **Design de Interfaces** entre o serviço de "Usuário" e o serviço de "Notificação" pode comprometer a **Independência de Deploy**.

Gabarito e Justificativa

Múltipla Escolha

1. **Resposta correta: b)**
    - *Justificativa:* Contratos claros (interfaces) permitem o desacoplamento. Se a interface é estável, o "como" o serviço funciona internamente pode mudar sem afetar os outros, facilitando a evolução contínua do sistema.
2. **Resposta correta: c)**
    - *Justificativa:* O roteamento é a função de "guarda de trânsito" do Gateway. Ele abstrai os endereços internos, oferecendo uma face única para o cliente e garantindo que cada mensagem chegue ao destino processador correto.

Questões Descritivas (Respostas Aceitáveis)

1. **Resposta esperada:** O aluno deve explicar que, sem observabilidade, seria impossível saber qual dos múltiplos serviços está causando a lentidão. Com o rastreamento, o arquiteto pode ver que o Gateway recebeu a chamada rápido, mas o serviço de "Notificação" demorou 10 segundos para responder, isolando o problema em um componente específico em vez de procurar em todo o sistema.
2. **Resposta esperada:** O aluno deve argumentar que se o serviço de "Notificação" mudar o formato de entrada de um dado (ex: mudar de ID para UUID) sem atualizar o contrato de interface, o serviço de "Usuário" deixará de funcionar ao tentar se comunicar. Isso quebra a independência, pois obriga que ambos os serviços sejam alterados e implantados simultaneamente para que o sistema volte a operar.

Exercício prático – Desafio: laboratório de **Sistemas Distribuídos**

Pré-requisitos para o ambiente do Laboratório de Redes e Cloud

Ambiente no Laboratório de Redes e Cloud

Arquivos de configuração (Dockerfiles)

Abaixo apresento os arquivos de configuração base para o seu laboratório de **Sistemas Distribuídos**, estruturados para que os alunos compreendam a separação de responsabilidades e o isolamento de ambiente.

Arquivos de Configuração (Boilerplate)

Para este exercício, os alunos devem criar uma estrutura de pastas simples:

- /lab-sd
    - /agendamento (contendo o app.py e Dockerfile)
    - /notificacao (contendo o server.js e Dockerfile)
    - docker-compose.yml

1. Microsserviço de Agendamento (Python)

Este serviço representa a camada de persistência e lógica de negócio central.

Dockerfile (/agendamento/Dockerfile):

```
Dockerfile
# Utilizando uma imagem leve de Python (Poliglotismo)
FROM python:3.9-slim

WORKDIR /app

# Simulação de dependências
RUN pip install flask

COPY . .

EXPOSE 8000

CMD ["python", "app.py"]
```

Código Base (/agendamento/app.py):

```
Python
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/agendar', methods=['POST'])
def agendar():
    # Simulação de lógica de agendamento
    return jsonify({"status": "Sucesso", "mensagem": "Consulta agendada no banco SQL"}), 201
```

```
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8000)
```

---

2. Microsserviço de Notificação (Node.js)

Este serviço demonstra a independência de stack, focado em tarefas assíncronas.

Dockerfile (/notificacao/Dockerfile):

```
Dockerfile
# Utilizando Node.js (Independência de Stack)
FROM node:18-alpine

WORKDIR /app

# Simulação de instalação de pacotes
RUN npm install express

COPY . .

EXPOSE 3000

CMD ["node", "server.js"]
```

Código Base (/notificacao/server.js):

```
JavaScript
const express = require('express');
const app = express();

app.get('/notificar', (req, res) => {
    // Simulação de envio de e-mail/SMS via Cache/NoSQL
    res.status(200).json({ servico: "Notificação", alerta: "Enviado com sucesso via Node.js" });
});

app.listen(3000, () => console.log('Serviço de Notificação rodando na porta 3000'));
```

---

3. Orquestrador (Docker Compose)

Este arquivo une os serviços no **Laboratório de Redes e Cloud** da São Judas.

docker-compose.yml:

```
YAML
version: '3.8'

services:
  # Serviço A: Agendamento (Python/Flask)
  api-agendamento:
    build: ./agendamento
    container_name: service-agendamento
    ports:
      - "8000:8000"
    networks:
      - rede-distribuida

  # Serviço B: Notificação (Node.js/Express)
  api-notificacao:
    build: ./notificacao
```

```
  container_name: service-notificacao
  ports:
    - "3000:3000"
  networks:
    - rede-distribuida

networks:
 rede-distribuida:
  driver: bridge
```

Guia de Execução para o Aluno

1. **Construção e Execução:** No terminal, dentro da pasta raiz, execute: docker-compose up --build.
2. **Verificação de Independência:** Acesse localhost:8000/agendar e localhost:3000/notificar simultaneamente.
3. **Teste de Resiliência:** Em um novo terminal, execute docker stop service-notificacao.
4. **Conclusão Crítica:** O aluno deve notar que, embora o serviço de notificação pare de responder, a API de agendamento continua operante, validando o conceito de **tolerância a falhas** por isolamento.

Roteiro de laboratório prático em Docker

Para materializar os conceitos de **Sistemas Distribuídos** que discutimos, preparei um roteiro de laboratório focado na transição do monólito para microsserviços, utilizando o **Docker** como ferramenta de isolamento e independência de deploy.

Este roteiro está alinhado com o cronograma de **Março**, focado em design de interfaces e componentização.

Lab: Decomposição e Isolamento com Docker

O objetivo deste laboratório é que o aluno saia da teoria e experimente, na prática, como o **Independência de Deploy** e o **Poliglotismo de Stack** funcionam na infraestrutura.

Contexto do Desafio

Imagine um sistema de "Gestão de Saúde". Inicialmente, ele é um monólito. Vamos separar a **Lógica de Agendamento** (focada em persistência) da **Lógica de Notificação** (focada em eventos), permitindo que cada uma rode em seu próprio container com tecnologias distintas.

1. Preparação do Ambiente

- **Requisito:** Docker e Docker Compose instalados no Laboratório de Redes e Cloud.
- **Conceito Chave:** O container garante que o serviço rode da mesma forma em qualquer lugar, eliminando o "na minha máquina funciona".

2. Passo a Passo Técnico

Passo A: O Microsserviço de Agendamento (Python + SQL)

Neste serviço, priorizamos a consistência dos dados. Utilizaremos Python para a API e um banco de dados relacional.

- **Tarefa:** Criar um Dockerfile que instale as dependências e exponha a porta 8000.
- **Destaque:** Este serviço é o "detentor da verdade" sobre os horários das consultas.

Passo B: O Microsserviço de Notificação (Node.js + NoSQL)

Aqui, o foco é a velocidade de entrega de mensagens. Como ele não precisa de relações complexas, utilizaremos Node.js e Redis para cache.

- **Tarefa:** Criar um segundo Dockerfile isolado.
- **Destaque (Poliglotismo):** Note que usamos linguagens diferentes para propósitos diferentes, mas ambos se comunicam via JSON/HTTP.

Passo C: Orquestração com Docker Compose

O aluno deverá criar um arquivo docker-compose.yml para subir ambos os serviços simultaneamente.

```
YAML
services:
  agendamento:
    build: ./agendamento
    ports: ["8000:8000"]
  notificacao:
    build: ./notificacao
    ports: ["3000:3000"]
```

3. Experimento de Tolerância a Falhas

Para exercitar o raciocínio crítico, peça aos alunos para:

1. Subir ambos os serviços.
2. Derrubar manualmente o container de notificacao (docker stop notificacao).
3. Tentar realizar um agendamento.

- **Resultado esperado:** O agendamento deve continuar funcionando, provando que a falha de um componente não derruba o ecossistema inteiro.

Exercícios de Fixação (Prática)

Questões de Múltipla Escolha

1. **Ao utilizar o Docker Compose para subir dois serviços (Agendamento e Notificação), qual característica fundamental de Sistemas Distribuídos estamos simulando?**
    - a) Centralização de logs em um único arquivo binário.
    - b) Isolamento de processos e independência de execução.
    - c) Eliminação da necessidade de protocolos de rede.

- d) Transformação de microsserviços em um monólito virtual.

2. **Se decidirmos atualizar a versão do Node.js apenas no serviço de Notificação, sem alterar o serviço de Agendamento, estamos aplicando o conceito de:**
    - a) Teorema CAP.
    - b) Sharding de banco de dados.
    - c) Independência de Deploy.
    - d) Computação de Borda (Edge Computing).

Questões Descritivas

1. Durante o laboratório, você derrubou o serviço de Notificação, mas o de Agendamento continuou ativo. Explique, tecnicamente, por que isso não seria possível em uma **Arquitetura Monolítica**.
2. Descreva como o uso de **Containers** facilita o **Poliglotismo de Stack** em uma equipe de desenvolvimento distribuída.

Gabarito e Justificativa

Múltipla Escolha

1. **Resposta correta: b)**
    - *Justificativa:* O Docker permite que cada serviço tenha seu próprio sistema de arquivos, bibliotecas e variáveis de ambiente, garantindo que um não interfira no outro, o que é a base do isolamento em SD.
2. **Resposta correta: c)**
    - *Justificativa:* A independência de deploy permite que o ciclo de vida tecnológico de um serviço seja gerido de forma autônoma, sem exigir que o sistema inteiro pare para uma atualização de versão de runtime.

Questões Descritivas (Respostas Aceitáveis)

1. **Resposta esperada:** Em um monólito, todos os módulos compartilham o mesmo espaço de memória e o mesmo processo do Sistema Operacional. Se o módulo de notificação causar um *memory leak* ou um erro não tratado, ele compromete todo o processo, derrubando o agendamento junto. No laboratório, o isolamento por containers impede essa propagação de erro.
2. **Resposta esperada:** O container "empacota" tudo o que o código precisa para rodar (bibliotecas Python, runtimes Node, etc.). Isso permite que o desenvolvedor do serviço A use Go e o do serviço B use Java sem que um precise instalar as ferramentas do outro em sua máquina, comunicando-se apenas através de contratos de interface (APIs).